Week 6

Gymnasium:
Famous Examples Only

From Conway to LangGraph: Agent Systems for Physicists in the LLM Era

CartPole · FrozenLake · Stable-Baselines3 · Reward Landscapes · Convergence

Università di Bologna · Dipartimento di Fisica

Where We Left Off

Week 5 recap — the math we now put to work

∇

Bellman Equation

$V(s) = \max_a [R(s,a) + \gamma \Sigma P(s'|s,a)V(s')]$ — the variational principle of RL

Q

Q-Table Agent

Pure NumPy Q-learning on a hand-coded GridWorld — no libraries, just the update rule

β

Exploration ↔ Exploitation

Temperature T in ε-greedy mirrors simulated annealing: $\beta = 1/T$ controls the balance

∞

Convergence guarantee

Q-learning converges to Q* under mild conditions (Watkins & Dayan 1992)

Why Gymnasium?

The standard interface that decouples agent from environment

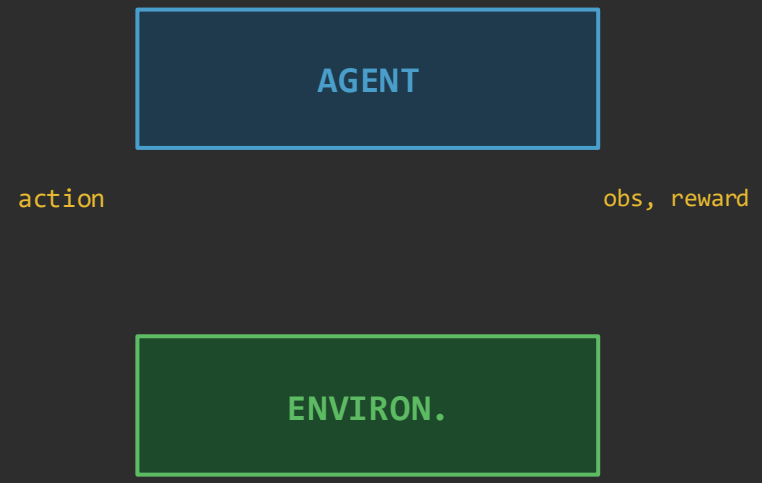
The Gym API in 5 lines

```
import gymnasium as gym
env = gym.make('CartPole-v1')
obs, info = env.reset()

for _ in range(1000):
    action = env.action_space.sample()
    obs, reward, terminated,
        truncated, info = env.step(action)
    if terminated: env.reset()

env.close()
```

The Agent-Environment Loop



env.observation_space	→	valid obs set
env.action_space	→	valid actions
env.step(a)	→	5-tuple

CartPole-v1

Continuous state · Dense reward · Classic control problem

The Physics

State vector (4D):

$x, \dot{x}, \theta, \dot{\theta}$ — position, velocity, angle, angular velocity

Action space:

Binary: push cart Left (0) or Right (1)

Reward:

+1 for every time step the pole stays upright

Termination:

$|\theta| > 12°$ or $|x| > 2.4$ m, or 500 steps

Equation of motion:

Coupled nonlinear ODE — standard inverted pendulum

The RL Framing

Variable	Range	Meaning
x	±4.8 m	Cart position
$\dot{x}$	$\pm\infty$	Cart velocity
θ	±0.418 r	Pole angle
$\dot{\theta}$	$\pm\infty$	Angular velocity

Why CartPole matters:

- Continuous state → can't use plain Q-table
- Dense reward → fast learning feedback
- Solvable in ~10 000 steps → great for demos
- PPO benchmark: 500/500 (perfect) in ~100k steps

FrozenLake-v1

Discrete state · Sparse reward · Stochastic transitions

4×4 Grid

S	F	F	F
F	H	F	H
F	F	F	H
H	F	F	G

START · SAFE · HOLE · GOAL

The RL Framing

State space:

16 discrete cells → Q-table is 16×4

Action space:

4 actions: L / R / U / D (slippery!)

Slipperiness:

P(intended) = 1/3; sideways = 2/3 — stochastic MDP

Reward:

R=1 at GOAL, R=0 everywhere else — sparse

Challenge:

Must traverse many steps with zero signal until G

Deterministic?:

gym.make('FrozenLake-v1', is_slippery=False) for easy mode

Two Environments, Two Physics Regimes

Choosing the right environment teaches you different things

Property	CartPole-v1	FrozenLake-v1
State space	$\mathbb{R}^4$ (continuous)	$\mathbb{Z}_{16}$ (discrete)
Action space	Binary {0,1}	4-directional {L,R,U,D}
Transitions	Deterministic ODE	Stochastic (slippery)
Reward	Dense (+1 every step)	Sparse (+1 only at goal)
Horizon	500 steps max	100 steps max
Solvable with	PPO / A2C (neural net)	Q-table (tabular RL)
Physics angle	Inverted pendulum / Lyapunov	Random walk on lattice

- *Together these two environments cover every combination of (discrete/continuous) × (sparse/dense) — the full parameter space of RL problems.*

Installation

Three packages — the entire RL stack

```
# Install
pip install gymnasium
pip install stable-baselines3[extra]
pip install shimmy
```

```
# Verify
import gymnasium as gym
import stable_baselines3 as sb3
print(sb3.__version__)  # ≥ 2.x
```

gymnasium

OpenAI Gym successor (Farama Foundation)
Provides CartPole, FrozenLake + 30+ environments
Stable API: obs_space, action_space, step(), reset()

stable-baselines3

Clean PyTorch RL implementations
PPO · A2C · DQN · SAC · TD3 ready to use
Consistent .learn() / .predict() API for all algos

shimmy

Compatibility shim for older Gym environments
Links atari-py, dm-control, MuJoCo to Gymnasium
Optional — needed only for non-standard envs

Stable-Baselines3: The Three-Line Promise

Train a state-of-the-art agent without touching algorithm internals

CartPole

```
from stable_baselines3 import PPO

model = PPO("MlpPolicy", env=gym.make("CartPole-v1"), verbose=1)
model.learn(total_timesteps=100_000)                    # ← train
model.save("cartpole_ppo")                                # ← save
```

FrozenLake

```
from stable_baselines3 import DQN

model = DQN("MlpPolicy", env=gym.make("FrozenLake-v1", is_slippery=False), verbose=1)
model.learn(total_timesteps=50_000)
```

PPO — What Is It Doing?

Proximal Policy Optimisation connects to Week 5 variational ideas

Policy Gradient in One Slide

Objective:

Maximize E[R] — total expected reward

Week 5 link:

Same Bellman principle, now as gradient ascent on θ

Actor:

$\pi_\theta(a|s)$ — policy network outputs action probabilities

Critic:

$V_\phi(s)$ — value network estimates state value

Clip trick:

L_{CLIP} prevents destructively large policy updates

Physics:

Analogy: constrained variational problem (KL budget)

SB3 Algorithms at a Glance

Algo	Type	Action	Use when
PPO	On-policy	Cont. & disc.	Default choice
A2C	On-policy	Cont. & disc.	Faster, noisier
DQN	Off-policy	Discrete only	FrozenLake
SAC	Off-policy	Continuous	Robotics
TD3	Off-policy	Continuous	Stable cont.

SB3 doc: stable-baselines3.readthedocs.io

The Physics Angle I: Reward Landscapes

Reward shaping as potential energy design

Reward as potential energy

The total discounted return $G_t = \Sigma \gamma^k r_{t+k}$ maps exactly to potential energy in a conservative field. The agent is a particle rolling toward the energy minimum (maximum reward). $\gamma < 1$ gives exponential distance weighting — identical to a Yukawa potential.

Reward landscape topology

Dense reward (CartPole) = smooth, convex landscape → easy gradient ascent. Sparse reward (FrozenLake) = flat with a single delta-peak at goal → near-zero gradient almost everywhere. This is why sparse rewards need exploration bonuses or reward shaping.

Reward shaping theorem (Ng et al. 1999)

Adding a potential-based shaping term $F(s,s') = \gamma\Phi(s') - \Phi(s)$ to the reward never changes the optimal policy. This is the RL analog of gauge invariance: we can shift the potential without altering the physics.

The Physics Angle II: Convergence & Stability

Learning curves as dynamical systems — does RL always converge?

Convergence Conditions

Tabular Q-learning:

Guaranteed if all (s,a) visited infinitely often and $\alpha_t \to 0$ at correct rate (Robbins-Monro)

DQN / PPO:

No global guarantee — empirical convergence only; replay buffer and target network help stability

Physics link:

Learning rate schedule $\alpha_t \sim 1/t$ mimics critical slowing-down near phase transitions

Oscillations:

Overfit, catastrophic forgetting, or limit cycles in policy space — analogous to chaos

CartPole: Lyapunov Stability

The trained CartPole controller keeps $(\theta, \dot{\theta})$ near the unstable equilibrium (0,0). We can verify this with a Lyapunov function:

```
V(s) = θ² + c·θ̇²  > 0
dV/dt = 2θ·θ̇ + 2c·θ̇·θ̈
# Stable if dV/dt < 0  →  θ̈ must push toward 0
```

A good PPO policy achieves near-Lyapunov stability empirically — $dV/dt < 0$ on average across episodes. We can measure this directly from rollouts!

- **Exercise 4 asks you to compute this.**

Visualising the Trained Agent

Render, record, and analyse — close the loop between code and intuition

Record video

```
from stable_baselines3.common.vec_env import VecVideoRecorder, DummyVecEnv

env = DummyVecEnv([lambda: gym.make("CartPole-v1", render_mode="rgb_array")])
env = VecVideoRecorder(env, "./videos", record_video_trigger=lambda step: step==0, video_length=500)
obs = env.reset()
for _ in range(500):
    action, _ = model.predict(obs)
    obs, _, done, _ = env.step(action)
```

Evaluate quantitatively

```
from stable_baselines3.common.evaluation import evaluate_policy

mean_reward, std_reward = evaluate_policy(
    model, gym.make("CartPole-v1"),
    n_eval_episodes=50, deterministic=True
)
print(f"Mean reward: {mean_reward:.1f} ± {std_reward:.1f}")  # 500.0 ± 0.0 = solved
```

Reading Learning Curves

Episode reward and entropy over training — a physicist's lens

TensorBoard / callback

```
from stable_baselines3.common.callbacks\
    import EvalCallback

eval_env = gym.make("CartPole-v1")
cb = EvalCallback(
    eval_env,
    best_model_save_path="./best/",
    eval_freq=5000, n_eval_episodes=10)
model.learn(total_timesteps=100_000,
          callback=cb)

# tensorboard --logdir ./tb_logs/
```

What the curves tell you

ep_rew_mean ↑

Primary signal — smoothed mean episode reward; should rise monotonically after warm-up

entropy_loss ↓

Policy entropy decreasing = agent becoming more deterministic = exploiting a learned strategy

value_loss ↓

Critic accuracy; high variance or oscillation signals unstable learning

approx_kl

Kullback-Leibler between old/new policy; PPO clips this to ≈ 0.01–0.05

explained_variance

R^2 of value predictions; < 0 = critic worse than random — needs more capacity

Analysing the Trained Policy

From black-box model to interpretable physics

CartPole — phase portrait of the trained policy

```
import numpy as np
import matplotlib.pyplot as plt

theta  = np.linspace(-0.2, 0.2, 50)
theta_dot = np.linspace(-2, 2, 50)
TH, TD = np.meshgrid(theta, theta_dot)

actions = np.zeros_like(TH)
for i, th in enumerate(theta):
    for j, td in enumerate(theta_dot):
        obs = np.array([[0, 0, th, td]], dtype=np.float32)  # x=0, xdot=0
        actions[j, i], _ = model.predict(obs, deterministic=True)
```

💡 **Physical insight:**

The decision boundary in (θ, θ) space is approximately linear — the PPO neural network rediscovers the classical LQR (Linear-Quadratic Regulator) controller for the inverted pendulum. The optimal gain ratio θ/θ tells you the relative weight placed on angle vs angular velocity — directly comparable to the LQR solution.

FrozenLake: Reading the Q-Table

Connecting DQN output to the Bellman analysis from Week 5

Extract Q-values from DQN

```
import torch
import numpy as np

# The DQN model stores a q_net — we extract Q(s,a) for all 16 states
states = torch.arange(16, dtype=torch.float32).unsqueeze(1)
with torch.no_grad():
    q_values = model.q_net(model.q_net.features_extractor(states))

q_np = q_values.numpy()  # shape (16, 4)

# Visualise as heatmap of best action per cell
best_actions = np.argmax(q_np, axis=1).reshape(4, 4)
action_labels = {0:'←', 1:'↓', 2:'→', 3:'↑'}
print('Greedy policy:')
for row in best_actions:
    print(' '.join([action_labels[a] for a in row]))
```

- Expected optimal path on deterministic grid: S → ↓ → → → → → G (row-major, avoiding holes)
- The Q-values encode exactly V*(s) from Week 5 — the Bellman values learned by iteration, now stored in a neural network.

Guided Exercises

Four problems — from running to researching

Ex 1

Run + Solve CartPole (30 min)

Install Gymnasium and SB3. Train a PPO agent on CartPole-v1 for 100 000 steps. Plot the learning curve (ep_rew_mean vs steps). Report the first timestep at which mean reward > 450. Compare to training with A2C — which converges faster?

Ex 2

FrozenLake: Sparse Reward Challenge (40 min)

Train DQN on FrozenLake-v1 with is_slippery=False (deterministic). Then switch to is_slippery=True (stochastic). Plot success rate vs training steps for both. Quantify the gap. Explain why sparse reward makes the stochastic case harder — connect to random walk physics.

Ex 3

Reward Landscape Visualisation (45 min)

For CartPole: sample 10 000 random trajectories. For each step compute the immediate reward gradient $\nabla_\theta R$. Plot the reward landscape as a function of pole angle θ. Compare to the gradient structure of a simple quadratic potential. Is it convex? Where is the maximum gradient?

Ex 4

★ Lyapunov Stability Test (60 min)

Load the trained CartPole PPO model. Run 100 episodes. For each timestep compute $V(s) = \theta^2 + \theta^2$. Plot $dV/dt = V(s_{t+1}) - V(s_t)$ as a function of θ. Show that $dV/dt < 0$ on average for $|\theta| < 0.1$ rad (stable region). Where does the policy fail Lyapunov stability?

References & Further Reading

Week 6

Gymnasium	Farama Foundation (2022). Gymnasium. https://gymnasium.farama.org — the official Gymnasium documentation. Successor to OpenAI Gym.
SB3	Raffin A. et al. (2021). Stable-Baselines3: Reliable Reinforcement Learning Implementations. JMLR 22(268), 1–8.
PPO	Schulman J. et al. (2017). Proximal Policy Optimization Algorithms. arXiv:1707.06347. The algorithm behind CartPole training.
DQN	Mnih V. et al. (2015). Human-level control through deep RL. Nature 518, 529–533. Original DQN for Atari — foundation of SB3 DQN.
Reward shaping	Ng A., Harada D., Russell S. (1999). Policy invariance under reward transformations. ICML 99, 278–287.
Lyapunov + RL	Berkenkamp F. et al. (2017). Safe model-based RL with stability guarantees. NeurIPS 2017.
LQR connection	Anderson B.D.O., Moore J.B. (2007). Optimal Control: Linear Quadratic Methods. Dover. — see Ch. 4 for inverted pendulum LQR solution.